

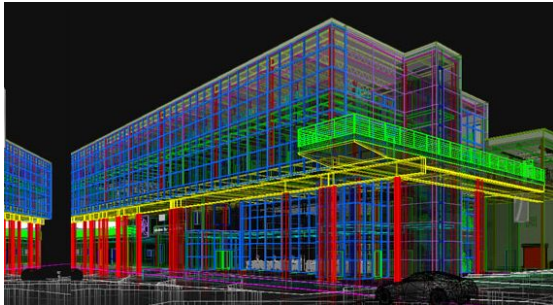
CHAPTER 13.

자바스크립트와 캔버스로 게임만들기



캔버스

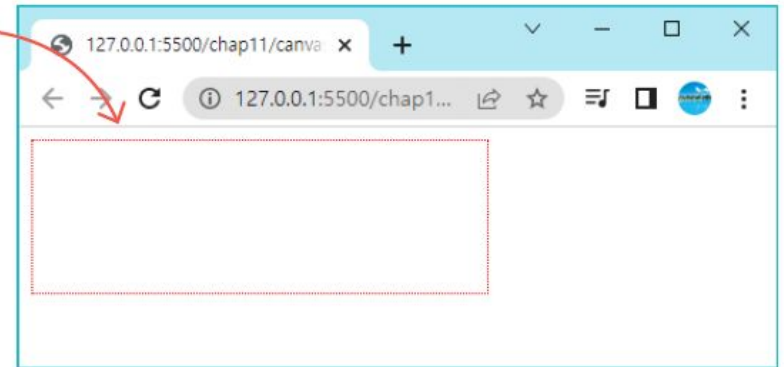
- 캔버스는 `<canvas>` 요소로 생성
- 캔버스는 **HTML** 페이지 상에서 사각형태의 영역
- 실제 그림은 자바스크립트를 통하여 코드로 그려야 한다.



캔버스 생성

- 캔버스는 `<canvas>` 요소로 생성된다. 캔버스는 HTML 페이지 상에서 사각 형태의 영역이다.

```
<canvas id="myCanvas" width="300" height="100"  
      style="border:1px dotted red;">  
</canvas>
```



컨텍스트 객체

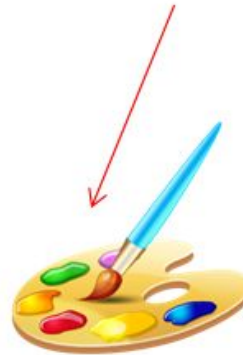
- 컨텍스트 (**context**) 객체 : 자바스크립트에서 물감과 붓의 역할을 한다.

```
var canvas = document.getElementById("myCanvas");  
var context = canvas.getContext("2d");
```

캔버스 요소

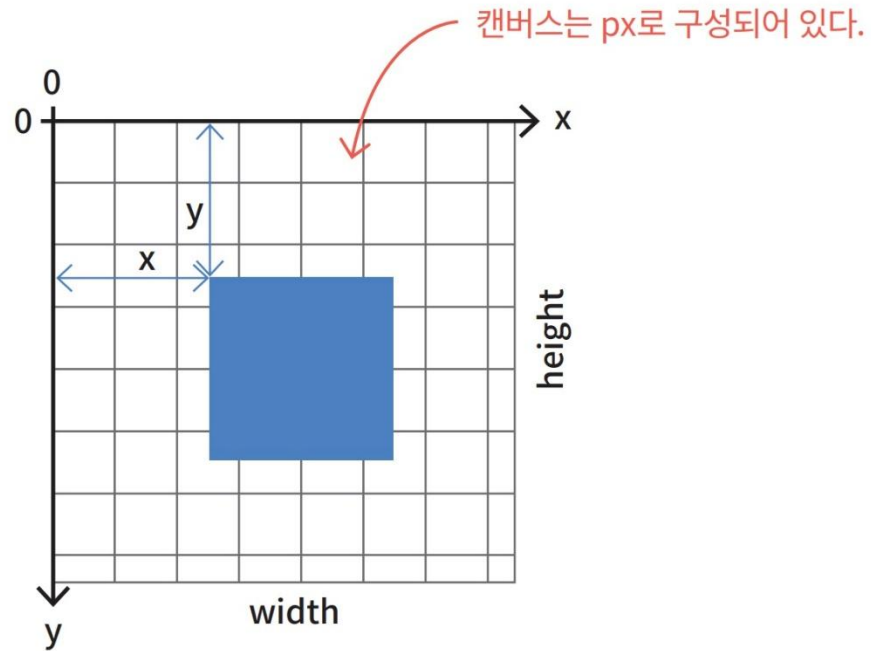


컨텍스트 객체



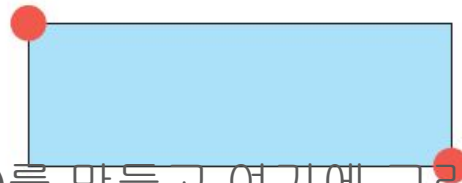
캔버스 좌표계

- 수학에서와는 다르게 캔버스의 좌측 상단의 좌표가 $(0, 0)$ 이다. 화면의 아래로 가면서 y 좌표는 증가한다.

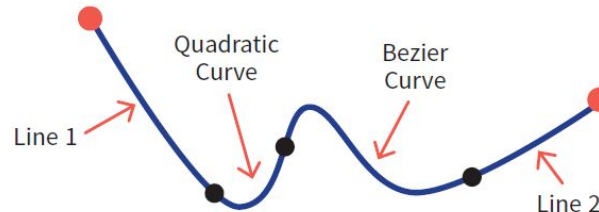


Canvas API

- 캔버스 요소를 통해 그래픽을 제공하는 라이브러리를 일반적으로 **Canvas API**라고 한다. **Canvas API**에서 그림을 그리는 방법에는 **2**가지가 있다.
- 첫 번째 방법은 도형을 캔버스에 직접 그리는 방법이다. 이 방법에서는 사각형만 그릴 수 있다. **fillRect()**나 **strokeRect()**를 이용한다.

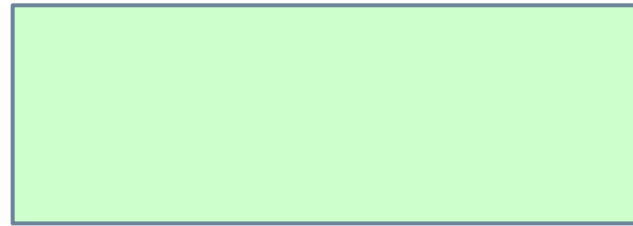


- 두 번째 방법은 비어있는 경로(**path**)를 만들고 여기에 그리기 명령들을 축적한 후에 한 번에 전부 캔버스에 그리는 방법이다.



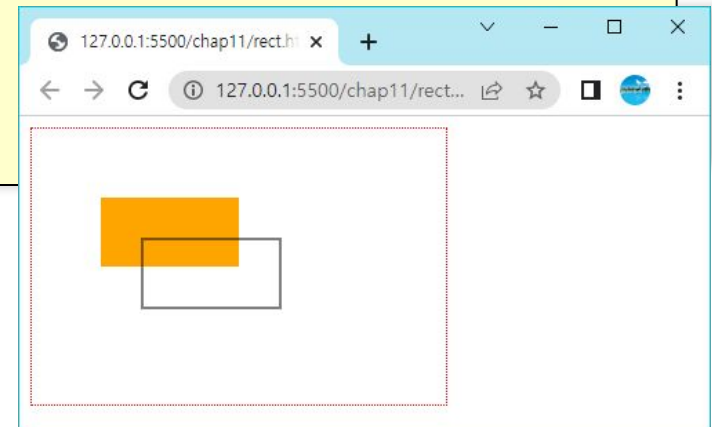
직사각형 그리기

- 직사각형 그리기는 <canvas>에서는 기본으로 제공하는 그리기 기능이다. 캔버스 위에 직사각형을 그리는 3가지 함수가 있다.
 - fillRect(x, y, width, height): 채워진 직사각형을 그린다.
 - strokeRect(x, y, width, height): 직사각형의 윤곽선만을 그린다.
 - clearRect(x, y, width, height): 특정 직사각형 영역을 지운다.



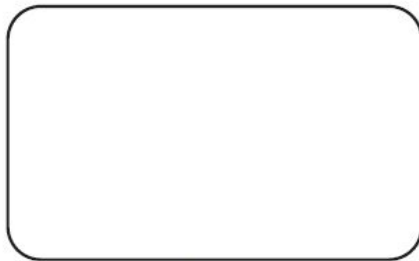
예제

```
<!DOCTYPE html>
<html>
<body>
  <canvas id="myCanvas" width="300" height="200"
    style="border: 1px dotted red"></canvas>
  <script>
    let canvas = document.getElementById("myCanvas");
    let context = canvas.getContext("2d");
    context.fillStyle = "orange";
    context.fillRect(50, 50, 100, 50);
    context.strokeRect(80, 80, 100, 50);
  </script>
</body>
</html>
```

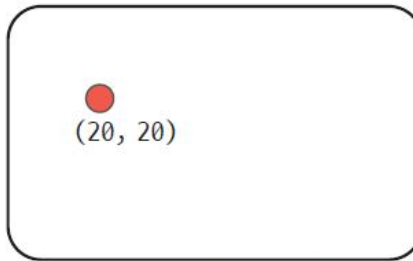


경로 그리기

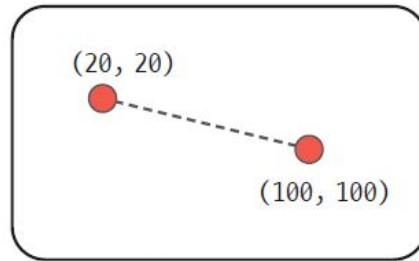
- Canvas API에서는 사각형 이외의 도형들은 모두 경로를 이용하여서 그려야 한다. 경로(path)는 기본적으로 점들의 집합이며, 도형과 곡선들이 포함된다.



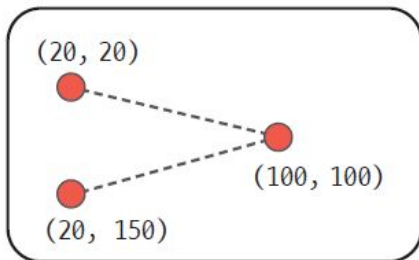
`context.beginPath();`



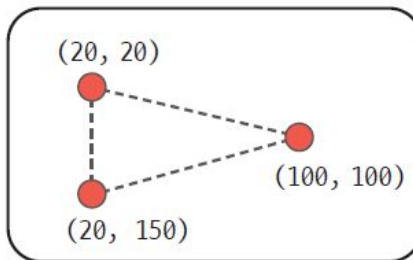
`context.moveTo(20, 20);`



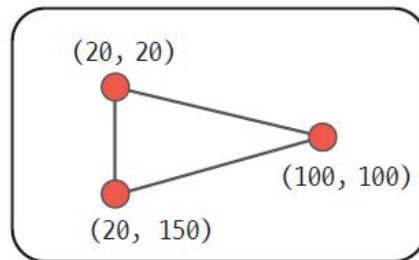
`context.lineTo(100, 100);`



`context.lineTo(20, 150);`



`context.closePath();`



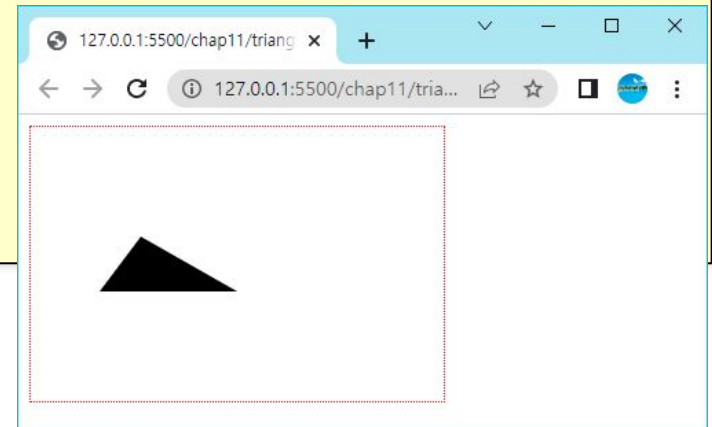
`context.stroke();`

경로에 사용되는 함수

- `beginPath()`: 새로운 경로를 만든다.
- `lineTo()`, `arcTo()`, ...: 경로에 도형을 그리는 명령을 추가한다.
- `moveTo()`: 펜을 든 후에 좌표 (x, y)로 이동한다.
- `closePath()`: 현재 경로의 시작 부분과 끝을 직선으로 연결하여 경로를 닫는다.
- `stroke()`: 경로에 포함된 도형의 윤곽선만을 그린다.
- `fill()`: 경로에 포함된 도형의 내부를 채워서 그린다.

예제

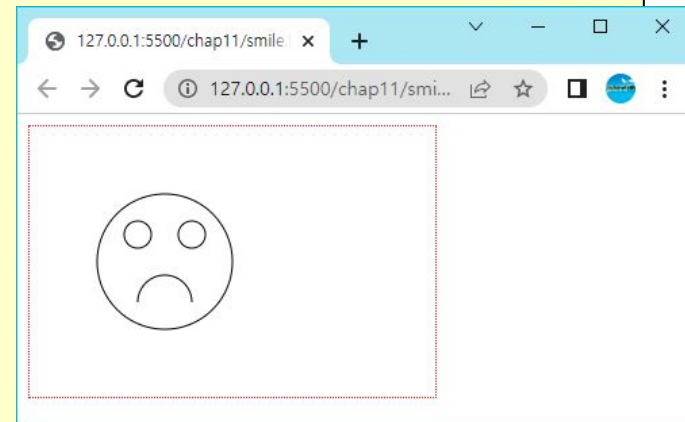
```
<!DOCTYPE html>
<html>
<body>
  <canvas id="myCanvas" width="300" height="200"
    style="border: 1px dotted red"></canvas>
  <script>
    let canvas = document.getElementById("myCanvas");
    let context = canvas.getContext("2d");
    context.beginPath();
    context.moveTo(80, 80);
    context.lineTo(50, 120);
    context.lineTo(150, 120);
    context.fill();
  </script>
</body>
</html>
```



펜 이동하기

- 실제로 어떤 것도 그리지 않지만, 아주 유용한 함수가 있다. 바로 `moveTo()` 함수가 있다. `moveTo(x, y)`: 펜을 들어서 좌표 (x, y)로 이동한다.

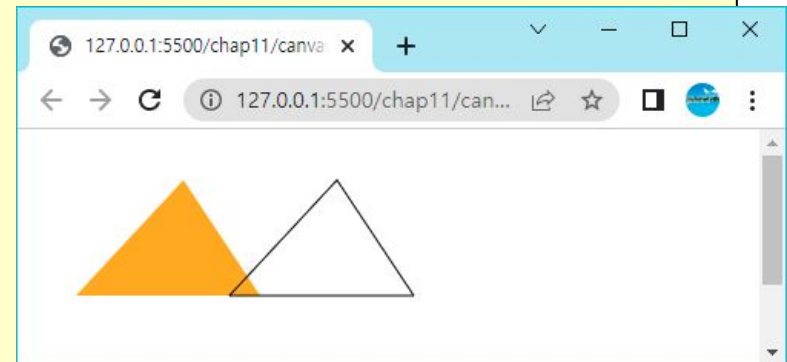
```
<!DOCTYPE html>
<html>
<body>
  <canvas id="myCanvas" width="300" height="200"
    style="border: 1px dotted red"></canvas>
  <script>
    let canvas = document.getElementById("myCanvas");
    let context = canvas.getContext("2d");
    context.beginPath();
    context.arc(100, 100, 50, 0, Math.PI * 2, true);
    context.moveTo(120, 130);
    context.arc(100, 130, 20, 0, Math.PI, true);
    context.moveTo(90, 80);
    context.arc(80, 80, 10, 0, Math.PI * 2, true);
    context.moveTo(130, 80);
    context.arc(120, 80, 10, 0, Math.PI * 2, true);
    context.stroke();
  </script>
</body>
</html>
```



직선그리기

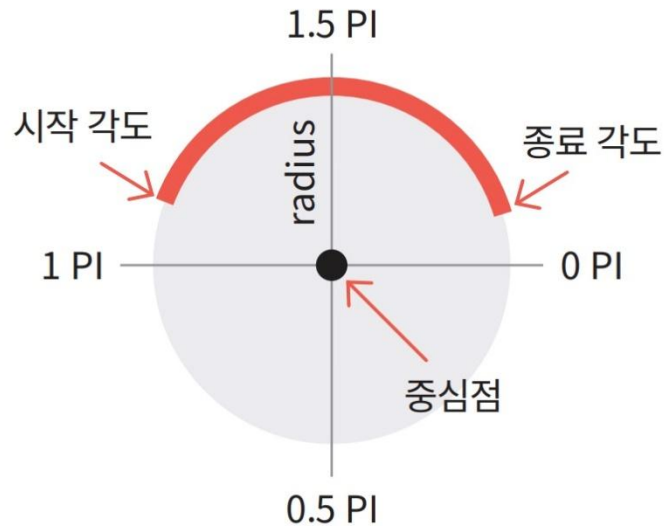
- 경로에 직선을 추가하려면 `lineTo()`를 사용할 수 있다. `lineTo(x, y)`: 현재의 위치에서 `x`와 `y`로 지정된 위치까지 선을 그린다.

```
<!DOCTYPE HTML>
<html>
<body>
  <canvas id="myCanvas" width="300" height="200"></canvas>
  <script>
    let canvas = document.getElementById('myCanvas');
    let context = canvas.getContext('2d');
    context.fillStyle = "orange";
    context.beginPath();
    context.moveTo(100, 25);
    context.lineTo(30, 100);
    context.lineTo(150, 100);
    context.fill();
    context.beginPath();
    context.moveTo(200, 25);
    context.lineTo(130, 100);
    context.lineTo(250, 100);
    context.closePath();
    context.stroke();
  </script>
</body>
</html>
```



원그리기

- 경로에 원을 추가하기 위해서는 다음의 메소드를 사용한다.
- `arc(x, y, radius, startAngle, endAngle, antiClockwise)` - (x, y) 를 중심으로 하여 반지름이 `radius`인 원을 그린다. `start`는 시작 각도이고 `end`는 종료각도이다.



원그리기

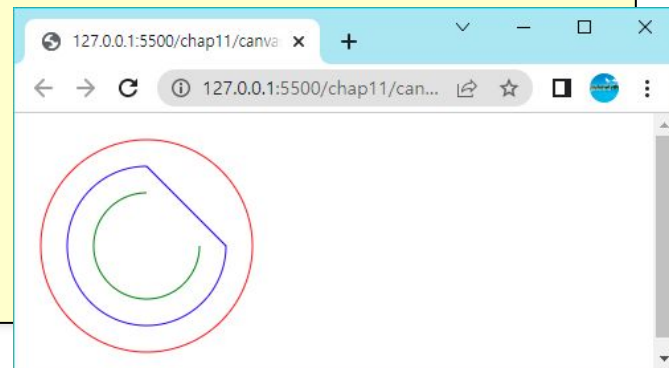
```
<script>
let canvas = document.getElementById('myCanvas');
let context = canvas.getContext('2d');

context.beginPath();
context.arc(100, 100, 80, 0, 2.0 * Math.PI, false);
context.strokeStyle = "red";
context.stroke();

context.beginPath();
context.arc(100, 100, 60, 0, 1.5 * Math.PI, false);
context.closePath();
context.strokeStyle = "blue";
context.stroke();

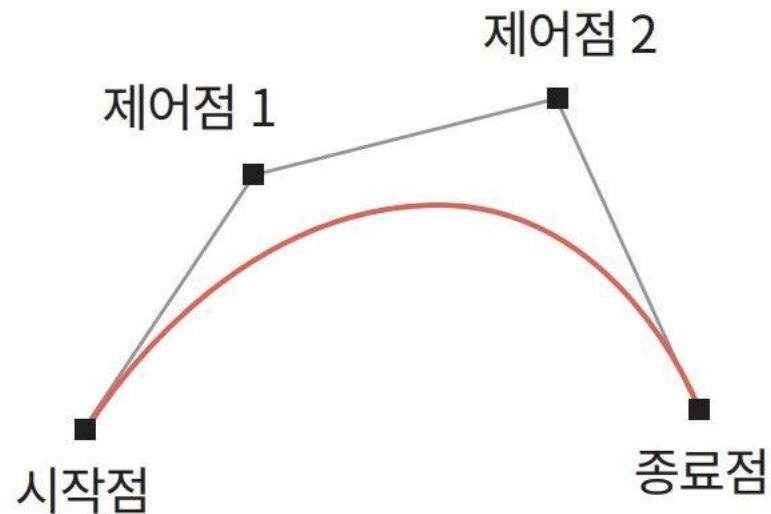
context.beginPath();
context.arc(100, 100, 40, 0, 1.5 * Math.PI, false);
context.strokeStyle = "green";
context.stroke();

</script>
```



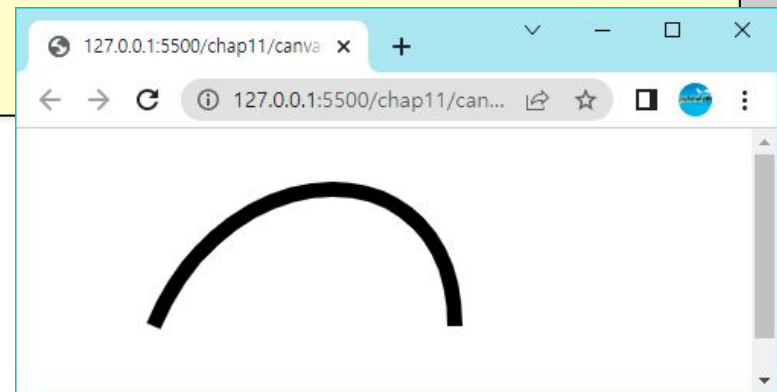
곡선그리기

- 베지어 곡선 같은 3차 곡선도 그릴 수 있다. 베지어 곡선은 4개의 제어점 안에 그려지는 3차 곡선이다.



곡선그리기

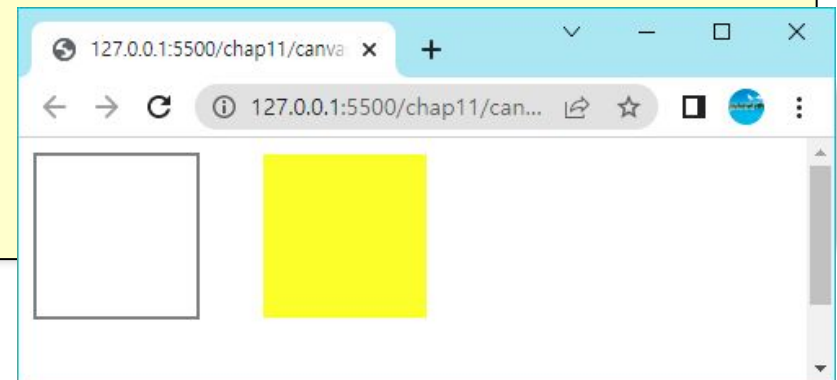
```
<script>  
  let canvas = document.getElementById('myCanvas');  
  let context = canvas.getContext('2d');  
  
  context.beginPath();  
  context.moveTo(90, 130);  
  context.bezierCurveTo(140, 10, 288, 10, 288, 130);  
  context.lineWidth = 10;  
  
  context.strokeStyle = 'black';  
  context.stroke();  
</script>
```



직사각형

- 경로에 직사각형도 추가할 수 있다. `rect()` 함수를 사용한다. 만약 채워진 사각형을 그리고 싶으면 `stroke()` 대신에 `fill()`을 호출하면 된다.

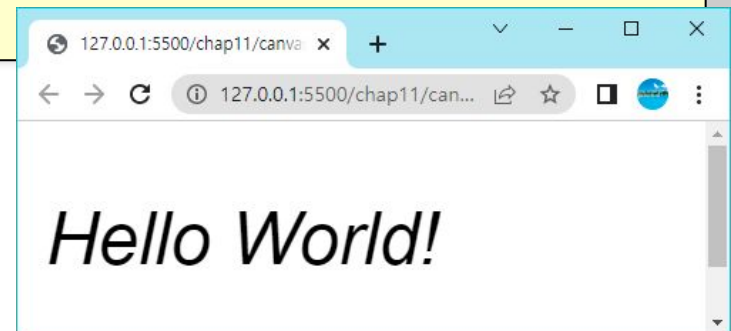
```
<body>
  <canvas id="myCanvas"width="300"height="200"></canvas>
  <script>
    let canvas = document.getElementById('myCanvas');
    let context = canvas.getContext('2d');
    context.beginPath();
    context.rect(10, 10, 100, 100);
    context.stroke();
    context.beginPath();
    context.rect(150, 10, 100, 100);
    context.fillStyle = "yellow";
    context.fill();
  </script>
</body>
```



텍스트 그리기

- font - 텍스트를 그리는데 사용되는 폰트 속성을 정의한다.
- fillText(text,x,y) - 캔버스에 채워진 텍스트를 그린다.
- strokeText(text,x,y) - 캔버스에 텍스트의 외곽선만을 그린다.

```
<script>  
  let canvas = document.getElementById('myCanvas');  
  let context = canvas.getContext('2d');  
  
  context.font = 'italic 38pt Arial'  
  context.fillText('Hello World!', 20, 100);  
</script>
```



이미지 그리기

- 캔버스에 이미지를 그리기 위하여 `drawImage(image, x, y)` 을 사용한다.

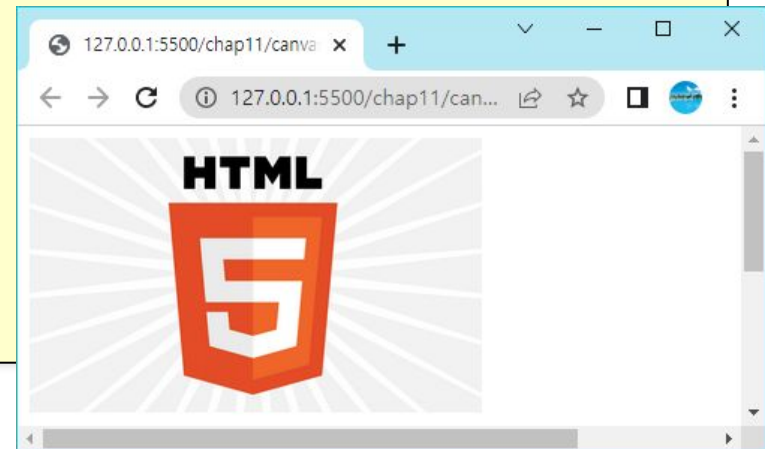
```
<body>
  <canvas id="myCanvas" width="600" height="400"></canvas>
  <script>
    let canvas = document.getElementById("myCanvas");

    let context = canvas.getContext("2d");
    let image = new Image();
    image.src = "html5_logo.png";

    image.onload = function () {
      context.drawImage(image, 0, 0);
    };

  </script>
</body>
```

...



색상

- 도형에 색을 적용하려면, 컨텍스트의 속성 `fillStyle`과 `strokeStyle`을 사용하면 된다.
 - `fillStyle = color`: 도형을 채우는 색을 설정한다.
 - `strokeStyle = color`: 도형의 윤곽선 색을 설정한다.

```
<!DOCTYPE HTML>
<html>
<body>
  <canvas id="myCanvas" width="600" height="300"></canvas>
  <script>
    let context = document.getElementById('myCanvas').getContext('2d');
    for (let i = 0; i < 6; i++) {
      for (let j = 0; j < 10; j++) {
        context.fillStyle = 'rgb(' + (255 - 20 * i) + ', ' +
          (255 - 20 * j) + ', 0)';
        context.fillRect(j * 50, i * 30, 45, 25);
      }
    }
  </script>
</body>
</html>
```



선 그리기 속성

- `context.lineWidth` – 선의 두께를 지정한다.
- `context.strokeStyle` – 선의 색상을 지정한다.
- `context.lineCap`– 양쪽 끝점의 모양을 지정한다. `butt`, `round`, `square` 중의 하나이다.



'butt' cap



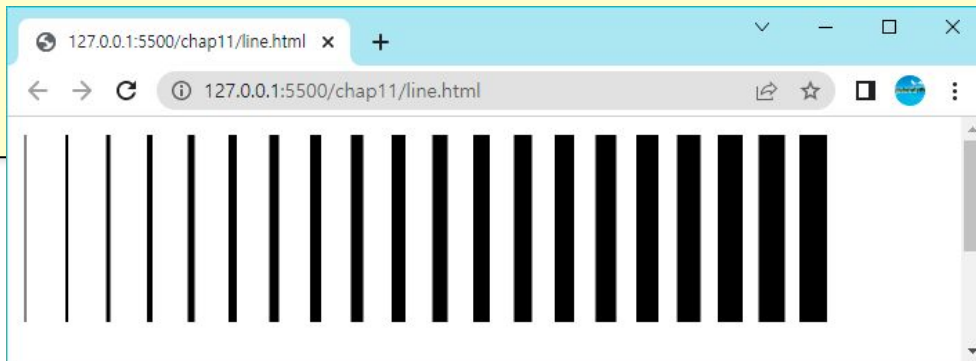
'round' cap



'square' cap

예제

```
<!DOCTYPE HTML>
<html>
<body>
  <canvas id="myCanvas" width="600" height="300"></canvas>
  <script>
    let context = document.getElementById('myCanvas').getContext('2d');
    for (let i = 0; i < 20; i++) {
      context.lineWidth = 1 + i;
      context.beginPath();
      context.moveTo(5 + i * 30, 10);
      context.lineTo(5 + i * 30, 200);
      context.stroke();
    }
  </script>
</body>
</html>
```

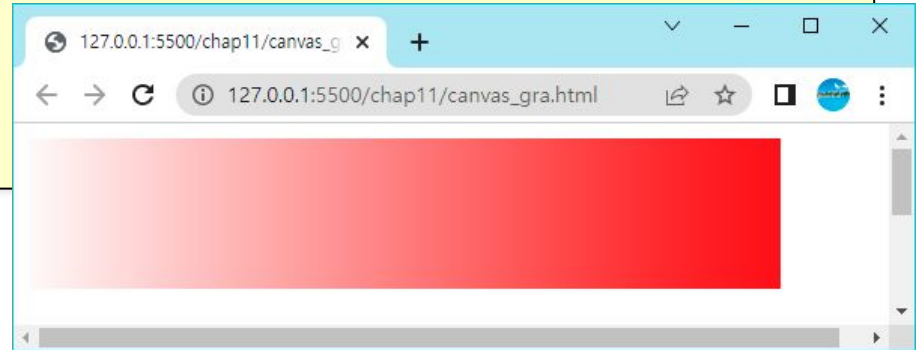


그라디언트

- `createLinearGradient(x, y, x1, y1)` - 시작점 좌표를 `(x1, y1)`로 하고, 종료점 좌표를 `(x2, y2)`로 하는 선형 그라디언트 객체를 생성한다.
- `createRadialGradient(x, y, r, x1, y1, r1)` - 반지름이 `r1`이고 좌표 `(x1, y1)`을 중심으로 하는 시작원과, 반지름이 `r2`이고 좌표 `(x2, y2)`를 중심으로 하는 종료원을 사용하여 원형 그라디언트 객체가 생성된다.
- `gradient.addColorStop(position, color)`: `gradient` 객체에 새로운 색상 종료 (color stop)을 생성한다. `position`은 0.0에서 1.0 사이의 숫자이고 그라디언트에서 상대적인 위치를 정의한다. `color` 인자는 색상을 나타내는 문자열이다. `position` 위치에 `color`를 지정한다.

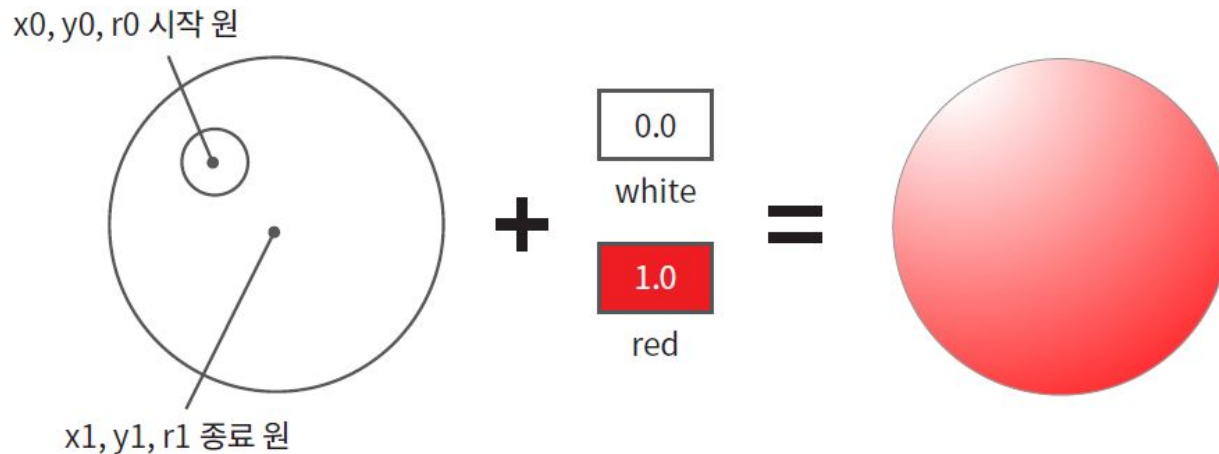
선형 그라디언트

```
<body>
  <canvas id="myCanvas" width="600" height="300"></canvas>
  <script>
    let canvas = document.getElementById('myCanvas');
    let context = canvas.getContext('2d');
    let gradient = context.createLinearGradient(0, 0, 500, 0);
    gradient.addColorStop(0, "white");
    gradient.addColorStop(1, "red");
    context.fillStyle = gradient;
    context.fillRect(10, 10, 500, 100);
  </script>
</body>
</html>
```



원형 그라디언트

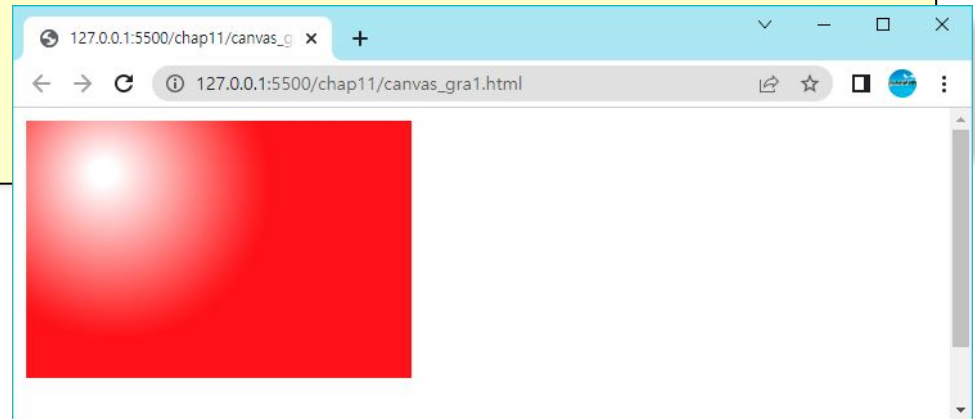
- `createRadialGradient(x0, y0, r0, x1, y1, r1)`: `x0, y0, r0`는 시작원의 좌표이고 `x1, y1, r1`는 종료원의 좌표이다.



예제

```
<body>
  <canvas id="myCanvas" width="600" height="300"></canvas>
  <script>
    let canvas = document.getElementById('myCanvas');
    let context = canvas.getContext('2d');
    let gradient = context.createRadialGradient(70, 50, 10, 80, 60, 120);

    gradient.addColorStop(0, "white");
    gradient.addColorStop(1, "red");
    context.fillStyle = gradient;
    context.fillRect(10, 10, 300, 200);
  </script>
</body>
```



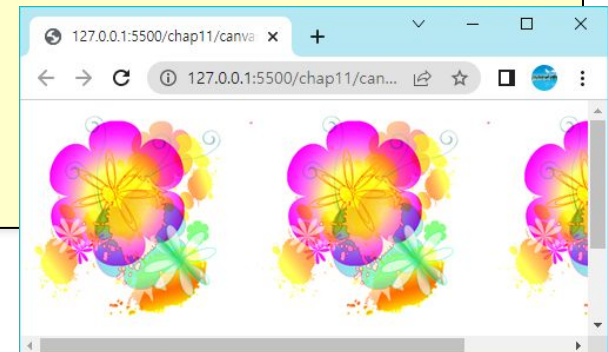
패턴 채우기

- 도형을 패턴으로 채워서 그리려면, 먼저 `createPattern()`을 이용하여서 패턴 객체를 생성한다. 이어서 컨텍스트의 `fillStyle` 속성을 패턴 객체로 설정한다. 최종적으로 `fill()` 메소드를 호출하여서 도형의 내부를 패턴으로 채우면 된다.

```
<script>
  let canvas = document.getElementById("myCanvas");
  let context = canvas.getContext("2d");

  let image = new Image();
  image.src = "pattern.png";
  image.onload = function () {
    let pattern = context.createPattern(image, "repeat");

    context.rect(0, 0, canvas.width, canvas.height);
    context.fillStyle = pattern;
    context.fill();
  };
</script>
```



도형 변환

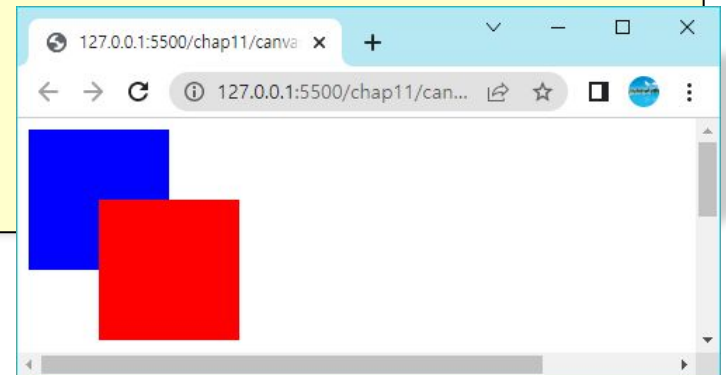
- 평행이동(translation)
- 신축(scaling)
- 회전(rotation)
- 밀림(shear)
- 반사(mirror)
- 행렬을 이용한 일반적인 변환

평행이동

```
<body>
  <canvas id="myCanvas" width="600" height="400"></canvas>
  <script>
    let canvas = document.getElementById('myCanvas');
    let context = canvas.getContext('2d');

    context.fillStyle = "blue";
    context.fillRect(0, 0, 100, 100);

    context.translate(50, 50);
    context.fillStyle = "red";
    context.fillRect(0, 0, 100, 100);
  </script>
</body>
```

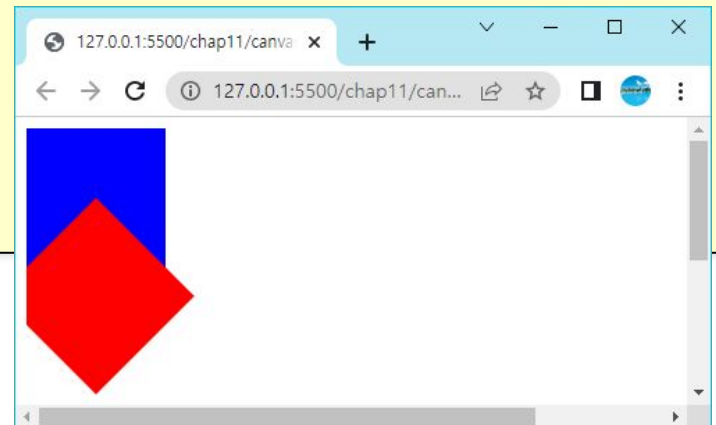


회전

```
<body>
  <canvas id="myCanvas" width="600" height="400"></canvas>
  <script>
    let canvas = document.getElementById('myCanvas');
    let context = canvas.getContext('2d');

    context.fillStyle = "blue";
    context.fillRect(0, 0, 100, 100);

    context.translate(50, 50);
    context.rotate(Math.PI / 4);
    context.fillStyle = "red";
    context.fillRect(0, 0, 100, 100);
  </script>
</body>
```

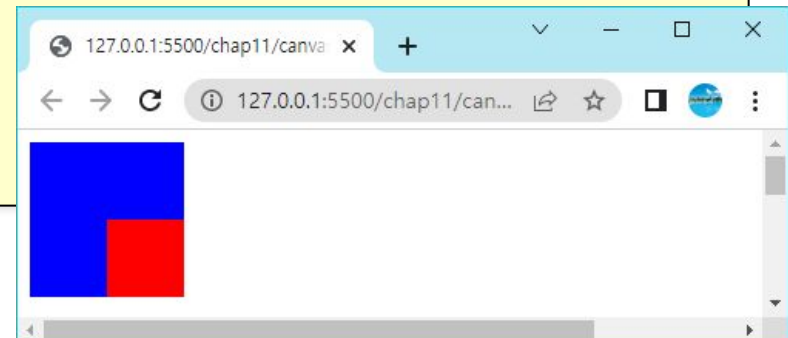


신축

```
<body>
  <canvas id="myCanvas" width="600" height="400"></canvas>
  <script>
    let canvas = document.getElementById('myCanvas');
    let context = canvas.getContext('2d');

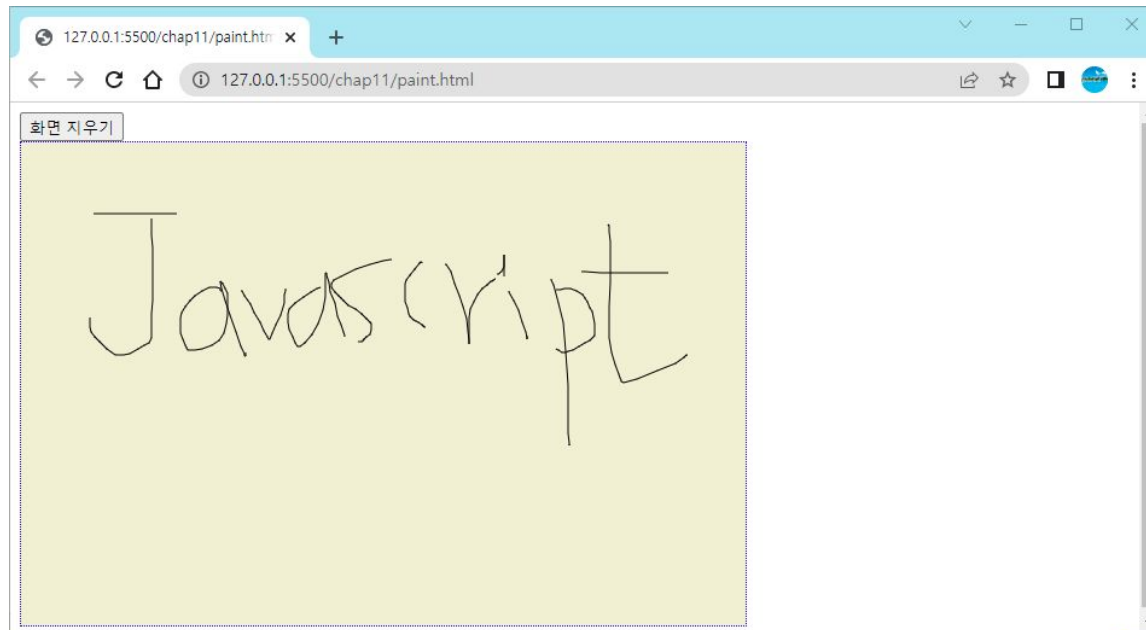
    context.fillStyle = "blue";
    context.fillRect(0, 0, 100, 100);

    context.translate(50, 50);
    context.scale(0.5, 0.5);
    context.fillStyle = "red";
    context.fillRect(0, 0, 100, 100);
  </script>
</body>
```



Lab: 그림판 만들기

- 자바스크립트만을 이용하여서 다음과 같은 기본적인 그림판 프로그램을 작성해보자. 마우스 이벤트와 그래픽을 결합해보자. “화면 지우기” 버튼을 클릭하면 화면이 지워져야 한다.



Sol.

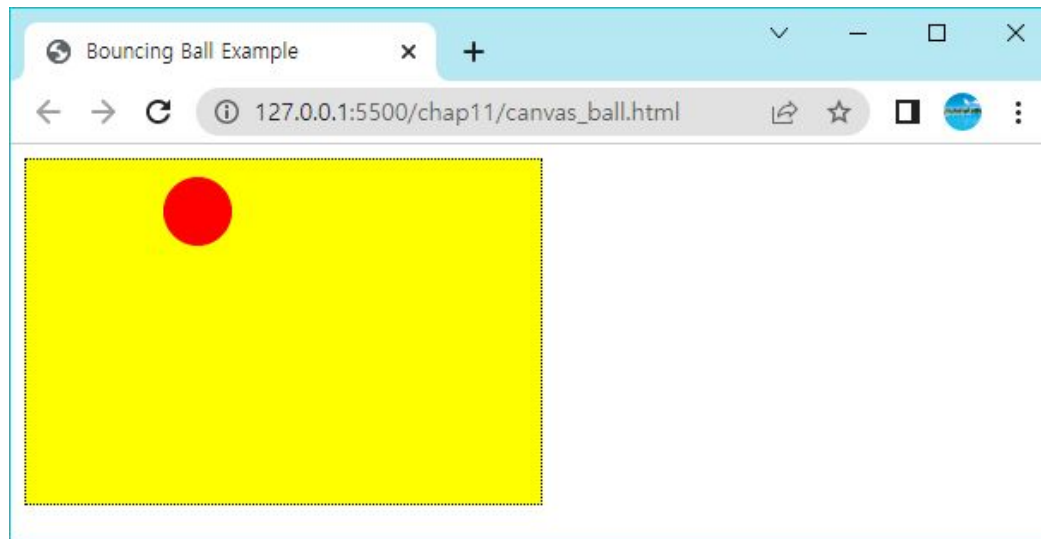
```
<!DOCTYPE html>
<html>
<style>
  #myCanvas {
    background-color:rgb(241, 239, 209);
    border: 1px dotted blue;
  }
</style>
<body>
  <button onclick="context.clearRect(0, 0, canvas.width, canvas.height);">화면 지우기 </button>
  <br>
  <canvas id="myCanvas" width="600" height="400"></canvas>
  <script>
    let canvas = document.getElementById("myCanvas");
    let context = canvas.getContext("2d");
    let last_x = 0, last_y = 0;
    let draw_mode = false;
```

Sol.

```
canvas.addEventListener("mousemove", function (event) {  
    if (!draw_mode) return;  
    let x = event.offsetX;  
    let y = event.offsetY;  
    context.lineTo(x, y);  
    context.stroke();  
    last_x = x;  
    last_y = y;  
});  
canvas.addEventListener("mousedown", function (event) {  
    last_x = event.offsetX;  
    last_y = event.offsetY;  
    context.beginPath();  
    context.moveTo(last_x, last_y);  
    draw_mode = true;  
});  
canvas.addEventListener("mouseup", function (event) {  
    draw_mode = false;  
});  
canvas.addEventListener("mouseout", function (event) {  
    draw_mode = false;  
});  
</script>
```

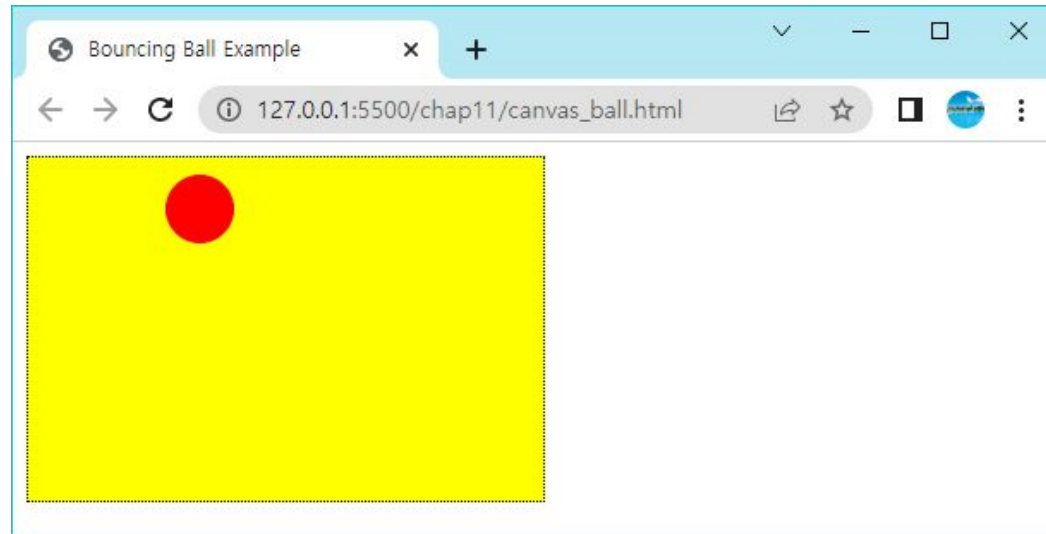
애니메이션: Bouncing Ball 예제

- 웹 페이지에서 애니메이션 효과를 내려면 많은 기법들이 있다. 제일 먼저 떠오르는 것은 **CSS3**를 이용한 애니메이션이다. 또 캔버스와 자바스크립트를 사용하는 애니메이션이 있다. 여기서는 후자의 경우를 살펴보자.



애니메이션 과정

1. 캔버스 지우기: `context.clearRect(0, 0, 300, 200);`
2. 캔버스에 그림 그리기: 이것은 앞에서 학습했던 대로 웹 페이지 안에 캔버스 요소를 생성하고 여기에 자바스크립트로 그림을 그리면 된다.
3. 위치를 업데이트한다.
4. 위의 절차를 반복한다.

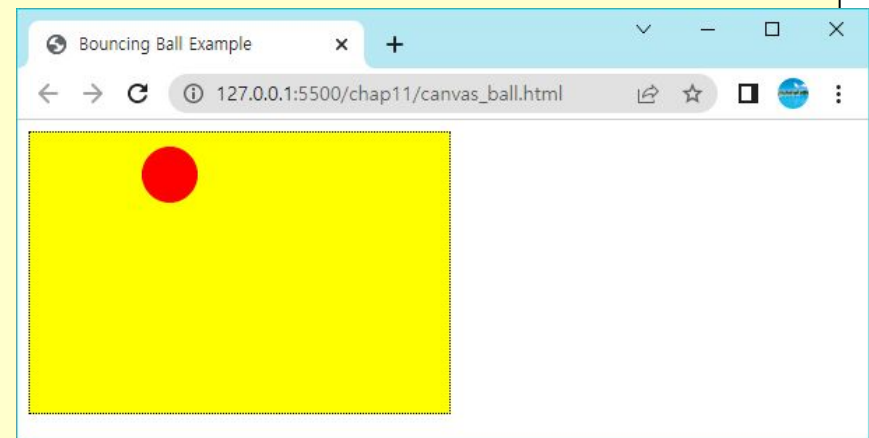


Sol.

```
<!DOCTYPE html>
<html>
<head>
  <title>Bouncing Ball Example</title>
  <style>
    canvas {
      background: yellow;
      border: 1px dotted black;
    }
  </style>
</head>
<body>
  <canvas id="myCanvas" width="300" height="200"></canvas>
  <script>
    let context;
    let dx = 5;
    let dy = 5;
    let y = 100;
    let x = 100;
```

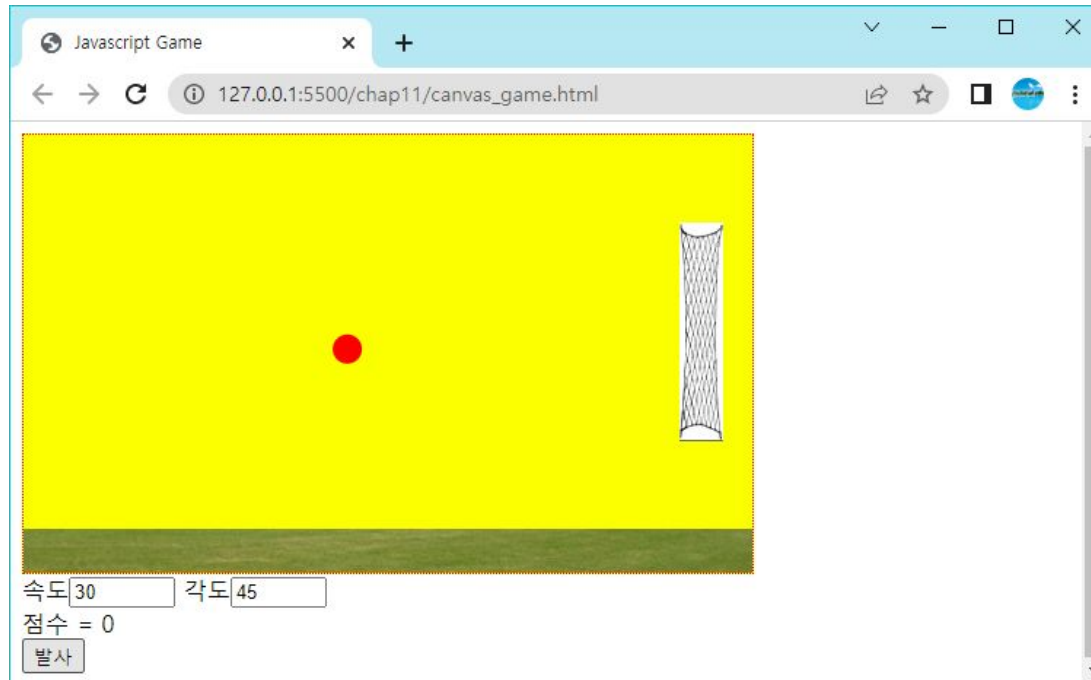
Sol.

```
function draw() {  
    let canvas = document.getElementById('myCanvas');  
  
    let context = canvas.getContext('2d');  
    context.clearRect(0, 0, 300, 200);  
    context.beginPath();  
    context.fillStyle = "red";  
    context.arc(x, y, 20, 0, Math.PI * 2, true);  
    context.closePath();  
    context.fill();  
    if (x < (0 + 20) || x > (300 - 20))  
        dx = -dx;  
    if (y < (0 + 20) || y > (200 - 20))  
        dy = -dy;  
    x += dx;  
    y += dy;  
}  
setInterval(draw, 10);  
</script>  
</body>  
</html>
```



간단한 게임 제작

- 앵그리 버드와 유사한 다음과 같은 게임을 제작



STEP #1

```
<html>
<head>
  <title>Javascript Game</title>
  <style>
    canvas {
      border: 1px dotted red;
      background-color: #fcff00;
    }
  </style>
</head>
```

```
<body>
  <canvas id="canvas" width="500" height="300"></canvas>
  <div id="control">
    속도<input id="velocity" value="30" type="number" min="0" max="100" step="1" />
    각도<input id="angle" value="45" type="number" min="0" max="90" step="1" />
    <div id="score">점수 = 0</div>
    <button>발사</button>
  </div>
</body>
</html>
```



STEP #2

```
<script>
  let image = new Image();
  image.src = "lawn.png";
  let backimage = new Image();
  backimage.src = "net.png";

  function drawBackground() {
    context.drawImage(image, 0, 270);
    context.drawImage(backimage, 450, 60);
  }

  function draw() {
    context.clearRect(0, 0, 500, 300);
    drawBackground();
  }

  function init() {
    context = document.getElementById('canvas').getContext('2d');
    draw();
  }
</script>
</head>
```



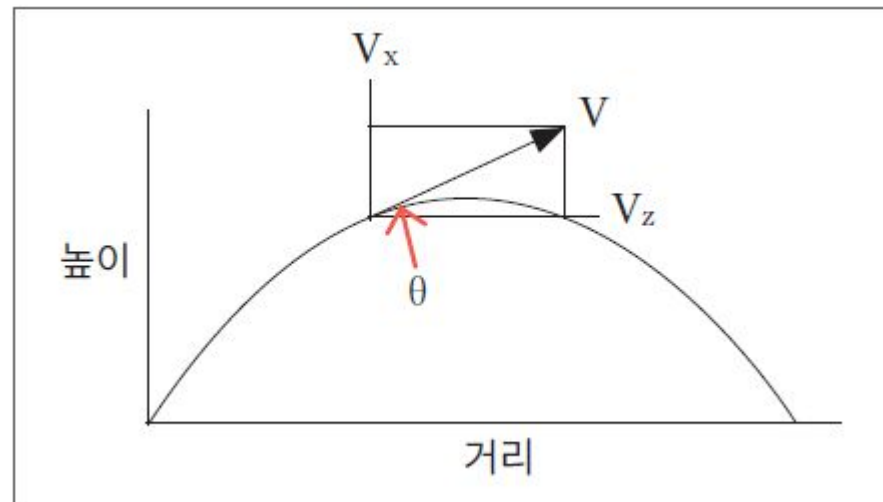
움직이는 공 만들기

- 공에 대한 많은 변수가 필요하다. 다음과 같은 변수를 선언하였다.
 - `let ballV;` - 공의 속도이다.
 - `let ballVx;` - 공의 x 방향 속도이다.
 - `let ballVy;` - 공의 y 방향 속도이다.
 - `let ballX;` - 공의 현재 x 좌표이다.
 - `let ballY;` - 공의 현재 y 좌표이다.
 - `let ballRadius;` - 공의 반지름이다.



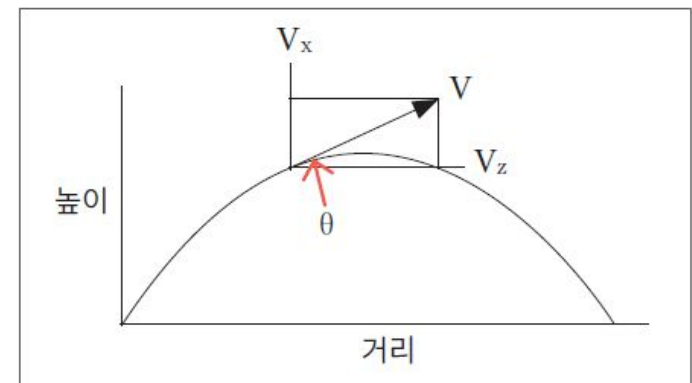
공의 움직임을 어떻게 계산하여야 하는가?

- 공의 y 방향 속도는 변하지 않는 것으로 가정한다. 물론 실제 상황에서는 공기의 저항을 받겠지만 이것은 무시하도록 하자. 공의 y 방향 속도는 중력 가속도 때문에 점점 느려질 것이다.
- $ball.V_x$; 공의 x 방향 속도로서 초기 속도에서 변하지 않는다.
- $ball.V_y$; 공의 y 방향 속도로서 초기 속도에서 중력 가속도 만큼 점점 느려진다.



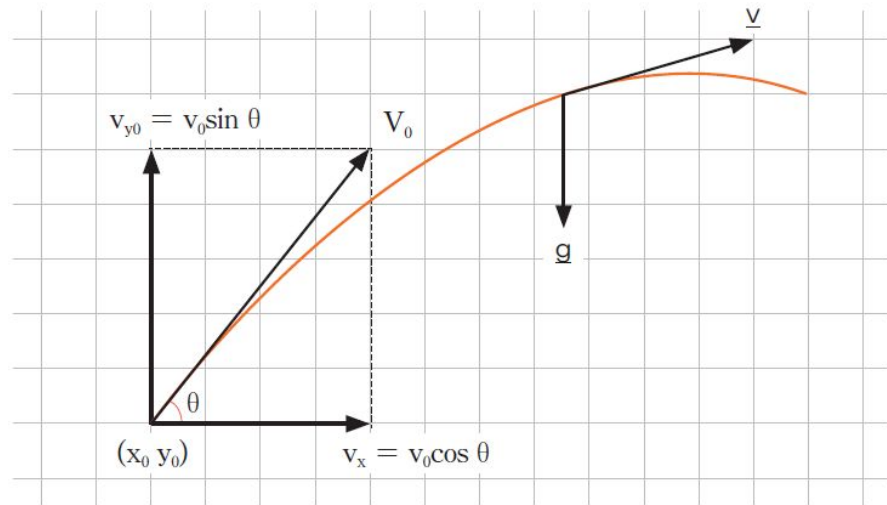
공의 움직임을 어떻게 계산하여야 하는가?

- 공의 속도 계산
 - $ballV_x = ballV_x;$
 - $ballV_y = ballV_y + 1.98;$
- 공의 현재 위치 계산
 - $ballX = ballX + ballV_x * 1.0 = ballX + ballV_x;$
 - $ballY = ballY + ballV_y * 1.0 = ballY + ballV_y;$



공의 초기 속도

- 공의 초기 속도는 사용자가 입력한 속도와 각도에 따라서 설정되어야 한다. 간단한 삼각함수를 사용하면 된다.
 - $\text{ballVx} = \text{velocity} * \text{Math.cos}(\text{angleR});$
 - $\text{ballVy} = -\text{velocity} * \text{Math.sin}(\text{angleR});$



전체 소스

```
<html>
<head>
  <title>Javascript Game</title>
  <style>
    canvas {
      border: 1px dotted red; /* 캔버스에 경계선을 그려준다. */
      background-color: #ffff00; /* 캔버스의 배경색을 지정한다. */
    }
  </style>
  <script>
    let context; /* 컨텍스트 객체 */
    let velocity; /* 사용자가 입력한 공의 초기속도 */
    let angle; /* 사용자가 입력한 공의 초기각도 */
    let ballV; /* 공의 현재 속도 */
    let ballVx; /* 공의 현재 x방향 속도 */
    let ballVy; /* 공의 현재 y방향 속도 */
    let ballX = 10; /* 공의 현재 x방향 위치 */
    let ballY = 250; /* 공의 현재 y방향 위치 */
    let ballRadius = 10; /* 공의 반지름 */
    let score = 0; /* 점수 */
    let image = new Image(); /* 이미지 객체 생성 */
    image.src = "lawn.png"; /* 이미지 파일 이름 설정 */
    let backimage = new Image();
    backimage.src = "net.png";
    let timer; /* 타이머 객체 변수 */
```

전체 소스

```
        /* 공을 화면에 그린다. */  
function drawBall() {  
    context.beginPath();  
    context.arc(ballX, ballY, ballRadius, 0, 2.0 * Math.PI, true);  
    context.fillStyle = "red";  
    context.fill();  
}  
  
        /* 배경을 화면에 그린다. */  
function drawBackground() {  
    context.drawImage(image, 0, 270);  
    context.drawImage(backimage, 450, 60);  
}  
  
        /* 전체 화면을 그리는 함수 */  
function draw() {  
    context.clearRect(0, 0, 500, 300);    /* 화면을 지운다. */  
    drawBall();  
    drawBackground();  
}  
  
        /* 초기화를 담당하는 함수 */  
function init() {  
    ballX = 10;  
    ballY = 250;  
    ballRadius = 10;  
    context = document.getElementById('canvas').getContext('2d');  
    draw();  
}
```


전체 소스

```
    /* 사용자가 발사 버튼을 누르면 호출된다. */
function start() {
    init();
    velocity = Number(document.getElementById("velocity").value);
    angle = Number(document.getElementById("angle").value);
    let angleR = angle * Math.PI / 180;

    ballVx = velocity * Math.cos(angleR);
    ballVy = -velocity * Math.sin(angleR);

    draw();
    timer = setInterval(calculate, 100);
    return false
}

    /* 공의 현재 속도와 위치를 업데이트한다. */
function calculate() {
    ballVy = ballVy + 1.98;

    ballX = ballX + ballVx;
    ballY = ballY + ballVy;

    /* 공이 목표물에 맞았으면 */
    if ((ballX >= 450) && (ballX <= 480) && (ballY >= 60) && (ballY <= 210)) {
        score++;
        document.getElementById("score").innerHTML = "점수=" + score;
        clearInterval(timer);
    }
}
```

전체 소스

```
        /* 공이 경계를 벗어났으면 */
        if (ballY >= 300 || ballY < 0) {
            clearInterval(timer);
        }
        draw();
    }
</script>
</head>

<body onload="init();">
    <canvas id="canvas" width="500" height="300"></canvas>
    <div id="control">
        속도<input id="velocity" value="30" type="number" min="0" max="100" step="1" />
        각도<input id="angle" value="45" type="number" min="0" max="90" step="1" />
        <div id="score">점수 = 0</div>
        <button onclick="start()">발사</button>
    </div>
</body>
</html>
```

Q & A

